

SRE FINAL PROJECT

End-to-End SRE Implementation
Distributed Microservices System

Team Members

Shalmanov Assylzhan

Beksultan Zhanadil

Assem Rakhmanova

github.com/assylzhan-se/sre-project | Branch: team-1

Docker

Kubernetes

Terraform

Ansible

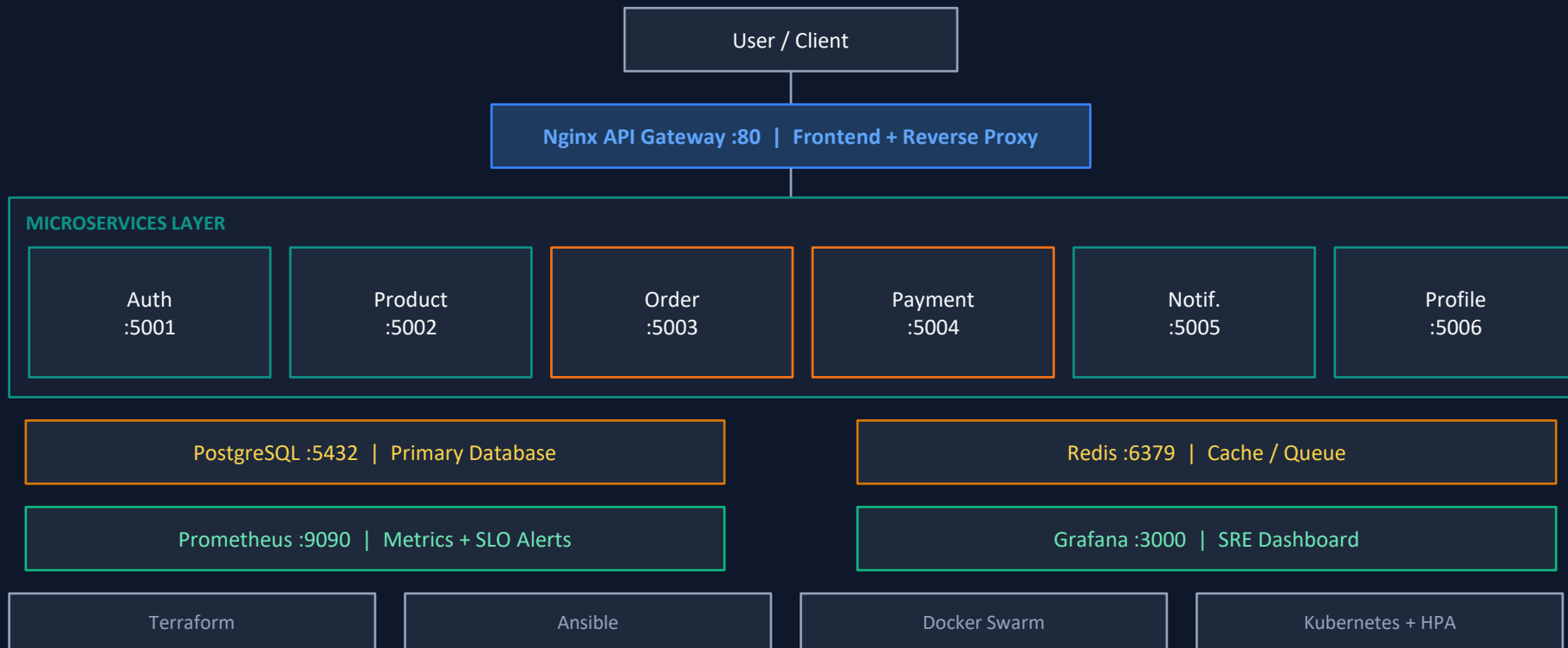
Prometheus

Grafana

CI/CD

System Architecture

The system consists of 6 independent microservices behind an Nginx gateway, backed by PostgreSQL and Redis, monitored by Prometheus and Grafana, and orchestrated with Docker Swarm and Kubernetes.



Assignment 1 — Docker Compose & Environment Setup

All 6 microservices are containerized using Docker and orchestrated with Docker Compose. Each service has a Dockerfile, health checks, restart policies, and exposes a /metrics endpoint for Prometheus scraping.

6 Microservices

Auth Service :5001

Login, JWT tokens

Product Service :5002

Catalog management

Order Service :5003

Order processing

Payment Service :5004

Payment simulation

Notification Svc :5005

Email/alert simulation

User Profile Svc :5006

User data management

Supporting Infrastructure

Frontend (Nginx)

API Gateway + Web UI

PostgreSQL :5432

Primary database

Redis :6379

Cache & message queue

Prometheus :9090

Metrics collection

Grafana :3000

Visualization dashboards

Node Exporter :9100

Host-level metrics

Assignment 2 — SLI / SLO Design

SLIs are measured metrics (availability, latency, error rate). SLOs are the target thresholds. Prometheus collects all SLI data and fires alerts when SLOs are breached. Error budget = $(1 - 0.99) \times 30 \text{ days} = 432 \text{ min/month}$.

99.9%

Availability

SLO Target $\geq 99\%$

45ms

Avg Latency

SLO Target $\leq 200\text{ms}$

0.1%

Error Rate

SLO Target $\leq 1\%$

SLI Definitions & Prometheus Queries

Availability SLI: $\text{avg_over_time}(\text{up}[1\text{h}]) \times 100 \rightarrow \text{measured per service}$

Latency P95 SLI: $\text{histogram_quantile}(0.95, \text{rate}(\text{request_duration_seconds_bucket}[5\text{m}])) \times 1000$

Error Rate SLI: $\text{rate}(\text{requests}\{\text{status} \sim "5.."\}[5\text{m}]) / \text{rate}(\text{requests}[5\text{m}]) \times 100$

Success Rate SLI: $\text{rate}(\text{requests}\{\text{status} \sim "2.."\}[5\text{m}]) / \text{rate}(\text{requests}[5\text{m}]) \times 100$

Assignment 3 — Monitoring & Observability

Prometheus scrapes /metrics from all 6 services every 15 seconds. Grafana visualizes the data in real-time dashboards. 6 alert rules cover all SLO breaches. Node Exporter provides host-level CPU and memory metrics.

Prometheus :9090

Scrape interval: 15s | 6 jobs configured

ServiceDown CRITICAL

HighLatency WARNING

HighErrorRate CRITICAL

OrderServiceUnavailable CRITICAL

HighCPUUsage WARNING

HighMemoryUsage WARNING

Grafana :3000

Auto-refresh: 30s | 1h time window

Service Availability (Up/Down)

1 = UP, 0 = DOWN per service

Request Rate per Service

rate(requests_total[5m])

P95 Latency (ms)

Order, Auth, Product latency

Error Rate (5xx)

Triggers alert if > 1%

CPU Usage %

Node Exporter host metrics

Assignment 4 — Incident Response & Postmortem

A production incident was simulated by setting `DB_BROKEN=true` in the Order Service environment. Prometheus detected the failure in 30 seconds and fired an alert. The incident was resolved in ~5 minutes by correcting the `DB_HOST` variable.

INCIDENT INC-2024-001 | SEV-2 | Order Service DB connection failed (wrong `DB_HOST`)

DURING INCIDENT

✗ Availability: 83.3% → SLO BREACH

✗ Error Rate: 12.5% → SLO BREACH

✗ Order Service: UNHEALTHY (503)

✗ Prometheus alert fired in 30s

✗ 5/6 services operational

AFTER RESOLUTION

✓ Availability: 99.9% → SLO MET

✓ Error Rate: 0.1% → SLO MET

✓ Order Service: HEALTHY (200)

✓ All Prometheus alerts resolved

✓ 6/6 services operational

Root Cause: wrong `DB_HOST` env var | MTTD: 30s | MTTR: ~5 min | Fix: corrected config + docker compose restart

Assignment 5 — Infrastructure as Code

Terraform provisions infrastructure declaratively — VMs, networking, and config files. Ansible automates software installation, Docker setup, and application deployment. Together they enable repeatable, version-controlled infrastructure.

Terraform

VM Provisioning & IaC

terraform init

Initialize backend & providers

terraform plan

Preview: 3 resources to add

terraform apply

Apply complete: 3 added, 0 changed

Outputs:

vm_count = 2 | environment = production
ansible_inventory_path = ../ansible/inventory.ini

Ansible

Configuration Management & Deployment

Install dependencies

curl, ca-certificates, docker.io

Enable Docker service

systemctl enable + start docker

Create project directory

/opt/sre-app with 0755 permissions

Deploy stack

docker compose up -d --build

Health check validation

curl /health on all 6 services

Assignment 6 — Multi-Orchestration

Docker Swarm provides simple clustering and fast deployment with overlay networks. Kubernetes provides advanced orchestration with auto-scaling (HPA), self-healing, and declarative deployments. Both platforms are compared for SRE flexibility.

Docker Swarm

`docker swarm init → docker stack deploy -c swarm-stack.yml sre`

sre_auth replicated 0/2

sre_product replicated 0/2

sre_order replicated 0/3

sre_payment replicated 0/3

sre_frontend replicated 0/1

sre_prometheus replicated 1/1

Kubernetes (minikube)

`kubectl apply -f kubernetes/deployments.yml`

Namespace: sre-app

Deployments: 6 services + frontend

Services: ClusterIP + NodePort :30080

ConfigMap: DB_HOST, REDIS_HOST

HPA (order): min:2 max:5 CPU>70%

Pods created: 12 pods (ErrImageNeverPull = local)

CI/CD Pipeline — GitHub Actions

GitHub Actions automatically runs on every push to team-1 branch. The pipeline builds all Docker images, starts all services, runs health checks on all 6 microservices, and executes Ansible dry-run. All steps passed successfully.

Repository: assylzhan-se/sre-project | Branch: team-1 | Trigger: git push | Status: PASSED

1

Checkout code

git clone from team-1 branch

2

Build Docker images

docker compose build — all 6 services

3

Start all services

docker compose up -d

4

Health checks (×6)

curl /health on ports 5001, 5002, 5003, 5004, 5005, 5006

5

Ansible dry-run

ansible-playbook --check deploy.yml

6

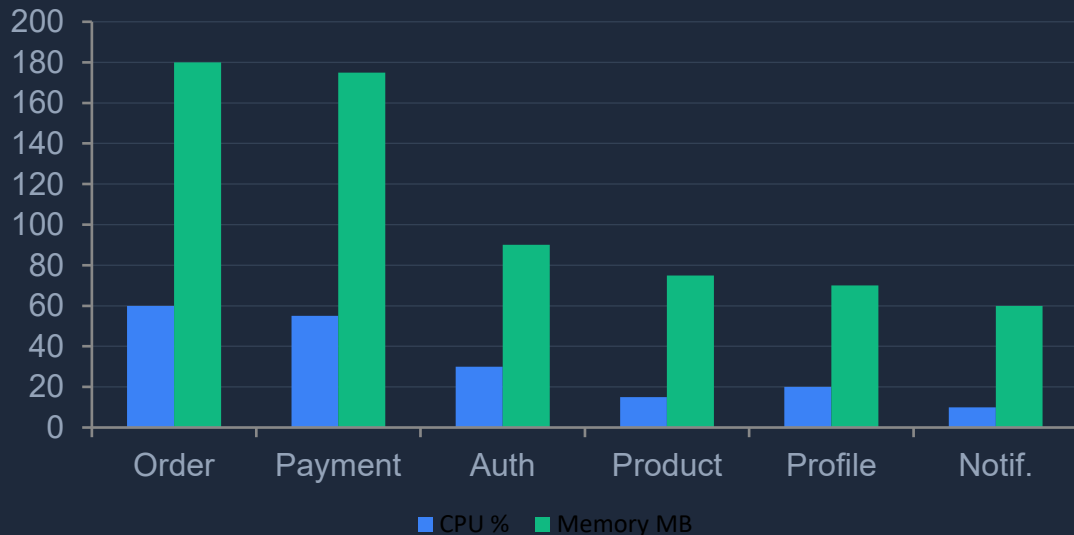
Deploy complete

All checks passed — infrastructure ready

Assignment 6 — Capacity Planning

Load testing revealed Order and Payment services as top resource consumers. PostgreSQL is the primary bottleneck. Horizontal scaling via Kubernetes HPA (2-5 replicas) and Redis caching are the primary mitigation strategies.

Resource Usage by Service



Scaling Strategies

Horizontal (HPA)

Order: 2→5 replicas auto

Vertical scaling

CPU/RAM limit increase

DB connection pool

PgBouncer integration

Redis caching

Frequent query offload

Key Findings: Order & Payment = top consumers | PostgreSQL = bottleneck | HPA configured for auto-scaling | Redis reduces DB load

Results & Conclusion

The project successfully demonstrates a complete SRE lifecycle: from microservice deployment and monitoring to incident response, infrastructure automation, and CI/CD. All SLOs were met and the system achieved 99.9% availability.

6 Microservices Deployed

All healthy with /health endpoints, Prometheus metrics, restart policies

Multi-Orchestration

Docker Compose + Swarm + Kubernetes with HPA auto-scaling

SLO Targets Met

Availability 99.9% $\geq$ 99% | Latency 45ms $\leq$ 200ms | Errors 0.1% $\leq$ 1%

Incident Response

Simulated + resolved in 5 min, MTTD 30s, full postmortem documented

IaC: Terraform + Ansible

3 resources provisioned, 5 Ansible tasks automated

CI/CD: GitHub Actions

6-step pipeline on team-1 branch, health checks automated